Python DB and Framework

- Introduction to embedding HTML within Python using web frameworks like Django or Flask
  - ✓ In frameworks like Django or Flask, Python is used for backend logic and HTML is used to display content. The framework connects both so users can see dynamic data on web pages

- Generating dynamic HTML content using Django templates
  - ✓ Django templates allow you to insert dynamic data into HTML using tags like `{{ }}`. This helps in showing different content based on user input or database data

- Integrating CSS with Django templates
  - ✓ CSS is used to style web pages and improve design. In Django, CSS files are linked using the `{% static %}` tag inside HTML templates

- How to serve static files (like CSS, JavaScript) in Django
  - ✓ Static files include CSS, JavaScript, and images. Django uses a static folder and proper settings to serve these files correctly

- Using JavaScript for client-side interactivity in Django templates
  - ✓ JavaScript is used to make web pages interactive, like handling clicks or animations. It works on the client side without reloading the page

- Linking external or internal JavaScript files in Django
  - ✓ JavaScript files can be linked using `{% static %}` or external URLs. This allows adding dynamic behavior to Django templates

- Overview of Django: Web development framework
  - ✓ Django is a powerful Python web framework used to build web applications quickly. It provides many built-in tools like authentication and admin panel

- Advantages of Django (e.g.,scalability,security)
  - ✓ Django is secure, scalable, and fast for development. It protects against common security issues and is suitable for large projects

- Django vs. Flask comparison: Which to choose and why
  - ✓ Django is a full-featured framework best for complex applications. Flask is lightweight and better for small or simple projects

- Understanding the importance of a virtual environment in Python projects
  - ✓ A virtual environment keeps project dependencies separate. It helps avoid conflicts between different Python projects

- Using venv or virtualenv to create isolated environments
  - ✓ You can create a virtual environment using `python -m venv`. It allows installing packages specific to one project only

- Steps to create a Django project and individual apps within the project
  - ✓ A Django project is created using `django-admin startproject`. Apps are created inside the project using `manage.py startapp`

- Understanding the role of manage.py, urls.py, and views.py
  - ✓ `manage.py` is used to run commands, `urls.py` handles routing, and `views.py` contains application logic

- Django's MVT (Model-View-Template) architecture and how it handles request-response cycles
  - ✓ MVT stands for Model, View, Template. It separates database, logic, and UI to handle requests and responses efficiently

- Introduction to Django's built-in admin panel.
  - ✓ Django provides a built-in admin interface for managing data. It allows easy control of database records without coding

- Understanding Django's ORM and how QuerySets are used to interact with the database

  Django ORM lets you interact with the database using Python code. QuerySets are used to retrieve and filter data easily

- Using Django's built-in form handling

- ✓ Django forms help collect and validate user input. They make handling data simple and secure

- Using AJAX for making asynchronous requests to the server without reloading the page
  - ✓ AJAX allows sending and receiving data without refreshing the page. It improves user experience and speed

- Techniques for customizing the Django admin panel
  - ✓ You can add filters, search, and custom display options. This makes the admin panel more powerful and user-friendly

- Introduction to integrating payment gateways (like Paytm) in Django projects
  - ✓ Payment gateways like Paytm allow secure online payments. They are integrated using APIs in Django projects

- Introduction to deploying Django projects to live servers like PythonAnywhere
  - ✓ Deployment means making your website live on the internet. Platforms like PythonAnywhere help host Django apps easily